

src\case_studies\D_mass\pid_controller.py

Generated: 2026-02-26 12:33:35

pid_controller.py

src\case_studies\D_mass\pid_controller.py

```
# 3rd-party
import numpy as np

# Local (controlbook)
from .. import common
from .. import params as P
from ..control.pid import PID
from ..control.dirty_derivative_filter import DirtyDerivativeFilter
from ..control import utils_design

class MassControllerPID(common.ControllerBase):
    def __init__(self):
        # TODO: where did old gains come from? Do we want to use them?
        # kp = 10.5
        # kd = 12.0
        # ki = 2.5
        # tuning parameters
        tr = 1.4
        zeta = 0.707
        ki = 2.5

        # system parameters
        b0 = P.tf_num[-1]
        a1, a0 = P.tf_den[-2:]

        # find gains
        alpha1, alpha0 = utils_design.get_des_CE(tr, zeta)
        kp = (alpha0 - a0) / b0
        kd = (alpha1 - a1) / b0
        self.z_pid = PID(kp, ki, kd, P.ts)
        print(f"{kp = :.2f}, {ki = :.2f}, {kd = :.2f}")

        self.F_eq = P.u_eq[0]

        # Dirty derivative filter for estimating zdot
        self.z_dot_filter = DirtyDerivativeFilter(P.ts, sigma=0.05)

    def update_with_state(self, r, x):
        raise NotImplementedError("This controller only uses measurement feedback.")

    def update_with_measurement(self, r, y):
        # unpack references and states
        z = y[0]
        z_ref = r[0]

        # Calculate derivative using dirty derivative filter
        z_dot = self.z_dot_filter.update(z)

        # z (modified) PID
        F_tilde = self.z_pid.update_modified(z_ref, z, z_dot)
        u = np.array([F_tilde + self.F_eq])
        u = self.saturate(u, u_max=P.force_max)
```

```
# include the following if you want to visualize dirty derivatives  
xhat = np.array([z, z_dot])  
  
return u, xhat.astype(np.float64)
```